

PINNED REPOSITORY AUDIT

AsymptoticAnalysis

after the eighteen reviews

Native-dispatch semantics, fresh defects,
and a bounded repair

With a comparison to built-in Wolfram Language capabilities

Repository	VladimirReshetnikov/Asymptotic
Audited revision	6687962f3c858a4f93623cfc496f33e35c6763d4
Review date	9 September 2026
Native observations	Wolfram Language 15.0.0, Linux x86-64
Deliverables	Article, candidate patcher, regression specifications, evidence

Evidence boundary

Three primary dispatch witnesses were reproduced in a real Wolfram kernel. Equivalent source edits were exercised on temporary standalone packages. The option-parser cost was measured by instrumenting a private predicate. The complete supplied regression file and the upstream suite are *not* claimed to have run. Python fixture checks and independent mathematical identities are recorded separately.

Contents

1	Conclusions and new findings	2
2	Snapshot, method, and exclusion of prior work	3
2.1	Pinned source and package identity	3
2.2	What was read and what was executed	3
2.3	How duplication was avoided	3
3	Architecture and comparison with Wolfram Language	4
3.1	The package is an analytic layer, not merely another Series head	4
3.2	Capability matrix	4
3.3	What should not be claimed about the built-ins	5
3.4	The meaningful distinction is the result contract	5
4	Mathematical examples that clarify the comparison	6
4.1	An exclusive cutoff and a native order are different requests	6
4.2	A generalized inverse with an independent coefficient oracle	6
4.3	Why the certificate implementation is substantively different	7
5	N01: configured backend defaults are ignored	8
5.1	Witness and observed result	8
5.2	Root cause	8
5.3	Bounded repair and its actual validation	9
5.4	The alias decision must remain visible	9
6	N02: selector discovery depends on unrelated input syntax	9
6.1	Minimal contrast	9
6.2	Why the literal and computed specifications diverge	10
6.3	Repair without releasing option values	10
6.4	Why appending all defaults is not a safe shortcut	11
7	N03: syntactic Function occurrence is not callable source role	11
7.1	Witness	11
7.2	The overbroad protection predicate	11
7.3	Narrow the Function test, not the branch policy	12
7.4	The right equivalence claim is deliberately limited	12
8	N04: quadratic validation of nested option containers	12
8.1	Source mechanism and exact recurrence	13
8.2	A linear structural plan	13
9	N05: a current README contradiction	14
10	Repair design and development opportunities arising from this audit	14
10.1	A small request record instead of another mathematical refactor	14
10.2	Make option ownership explicit	15
10.3	Metamorphic tests for request semantics	15
10.4	Focused acceptance and performance workloads	15
10.5	Priorities	16
11	Artifacts, execution evidence, and reproduction	16
11.1	What the archive contains	16
11.2	Stage and test a temporary standalone candidate	17
11.3	Reproduce the independent checks and article	17
11.4	Final assessment	17
	References	18

1 Conclusions and new findings

The most useful incremental audit target at the pinned revision is the boundary between the package’s analytic expansion engines and its newly integrated native backends. The mathematical engines and their outstanding obligations have already received eighteen reviews. Repeating their lists would obscure what changed. This report instead identifies three reproducible dispatch defects, one precisely characterized parser-cost issue, and one concrete current-documentation contradiction.

The principal result is not that the package produces a newly discovered false asymptotic theorem in these examples. It is that apparently equivalent ways of expressing the same request select different contracts, retain different orders, or fail unnecessarily. This matters because native and package results are intentionally *not* interchangeable: their order conventions, representations, and available analytic evidence differ.

ID	Finding	Evidence	Priority
N01	The dispatcher ignores a configured "Backend" default; the alias’s separate option table is also ineffective.	Native baseline and primary-default candidate observations.	High within compatibility work: changes selected contract and order.
N02	A computed option key can escape selector stripping. An alias for "Backend" fails with a literal specification but works with a computed one.	Native baseline and candidate observations.	Medium: valid programmatic calls become syntax-dependent.
N03	A <code>Function</code> anywhere inside a source program is treated as a protected callable, even after application has produced a scalar expression.	Native baseline and candidate observations.	Medium: unnecessary loss of native coverage.
N04	Nested option containers are repeatedly revalidated during selector discovery, producing quadratic predicate work.	Source recurrence and native entry-counter measurements.	Targeted performance repair; no general speed ranking asserted.
N05	The README’s final development-status paragraph contradicts its introduction and the implemented automatic routing.	Direct documentation/source comparison.	Low, inexpensive, and worth correcting with the code.

These are deltas within an existing compatibility program, not a claim that native interoperability itself is a new recommendation. In particular, register items B01–B04 already describe the required native input-superset goal, and P06/P08 already discuss resource accounting and profiling in general. The contribution here is the specific counterexamples, causes, measured parser recurrence, and narrowly scoped repairs. The supplied novelty ledger makes those relationships explicit.

The bounded candidate fixes N01 for the primary function, N02’s selector-key path, and N03’s callable classification. It intentionally leaves the independent configuration semantics of `AsymptoticExpand` undecided and does not claim to fix N04. A configured primary default is inherited by the alias after the candidate change; an independently configured alias default still needs an ownership decision. This distinction is important when judging the patch.

2 Snapshot, method, and exclusion of prior work

2.1 Pinned source and package identity

Every repository claim refers to commit 6687962f3c858a4f93623cfc496f33e35c6763d4. The repository is still named `Asymptotic`, but its current package and public context are `AsymptoticAnalysis`. The main public function names, including `AsymptoticInverse`, remain in use. The root standalone file is generated from modular sources; it is not the canonical place to maintain a repair. The README and kernel source map describe that organization [1, 2].

The commit timestamp is 10 September 2026 at 00:18:17 UTC, which is 9 September in the user’s Pacific time zone. The review date on the title page therefore does not indicate a discrepancy between the snapshot and the local date. The native evaluator reported 15.0.0 for Linux x86 (64-bit) (May 6, 2026). The repository’s Windows 15.0.1 validation records belong to their own runs and are not reused as execution evidence for this audit.

2.2 What was read and what was executed

The exclusion baseline is the maintained implementation register, its complete 123-entry finding crosswalk, and both review-wave indexes. The current native-contract document, main README, kernel map, dispatcher, ordinary arithmetic dispatcher, certificate implementation, relevant public declarations, and selected native tests and validation scripts were inspected directly. The architecture and specialized-function survey also uses the repository’s maintained documentation. This is not a claim to have independently proved every theorem or reread every page of all eighteen historical articles.

The deepest line-by-line review in this pass concerns `src/Kernel/NativeCompatibility.wl`, `src/Kernel/SeriesArithmetic.wl`, and `src/Kernel/InverseCertificates.wl`. The public declarations and assumption boundary in `src/Kernel/AsymptoticAnalysis.wl` provide the surrounding contract. The ordinary arithmetic and certificate inspection helps distinguish a dispatcher failure from an error in the mathematical engine; it did not produce a new demonstrated mathematical counterexample in this pass.

The observations were made through a real Wolfram kernel loading the pinned standalone with `Get[URLDownload[...]]`. Source access was through the GitHub connector, not a local full repository checkout. Successful native responses are transcribed in `evidence/native_observations.json`; that file is not presented as a raw complete kernel log. Some later service calls, including an attempted selected upstream test report, failed at the connector level and returned no usable test result. Those failures establish neither a package failure nor a passing test count.

2.3 How duplication was avoided

The wave-1 reviews use snapshots 07a9781 or 75de875; wave 2 uses 921387e. Their indexes explicitly distinguish historical findings, proposed patches, native observations, and independent model checks [8, 9]. The register marks a number of subsequent repairs as focused-verified and others as pending or merely audit candidates.

This report does not reopen those items under new labels. In particular, it does not present the already recorded remainder, exponent-grouping, nonuniform-composition, refinement-demand, native-index, flat-sector, numerical conditioning, or certificate-accuracy issues as discoveries. It also does not repackage the existing mathematical-extension roadmap as a new development proposal. Those obligations remain in the register. The new roadmap below is restricted to the request-language defects demonstrated here.

A broad pre-existing item is not evidence that every later concrete failure was already reported. N01–N03 sharpen B01/B02 with reproducible failures in particular source paths. N04 identifies a specific repeated-validation mechanism rather than repeating “profile the package.” N05 names the exact stale paragraph instead of alleging that documentation is generally missing.

3 Architecture and comparison with Wolfram Language

3.1 The package is an analytic layer, not merely another Series head

The repository combines several responsibilities. The main kernel implements real power-log models, sparse coefficient operations, inverse coefficient calculus, result construction, and module loading. Source-coordinate modules translate finite and infinite endpoints into local charts. Inverse-expression modules recognize held function syntax and branch restrictions. Core, Lambert, Gamma, and Barnes modules provide specialized inverse constructions. Other adapters admit particular special-function expansions. Explicit series operations and ordinary arithmetic preserve compatible remainder contracts; composite envelopes cover some cases outside a common ordered representation. Refinement stores or replays construction information. Finally, numerical inverse checks and exact interval certificates are separate APIs [2, 3, 6, 7].

The native compatibility layer sits *before* some real-model admission checks. It can return a `GeneralizedSeries` wrapper with “Kind” $\rightarrow$ “Native” and preserve the complete native expression. It cannot turn a formal native order into an independently established analytic remainder. This separation is an important strength of the design. A complex native result is not silently promoted to the package’s real analytic domain [5].

3.2 Capability matrix

The following comparison separates documented capabilities, repository interfaces, and actual observations. It does not claim that every special function was differentially tested or that one engine is universally faster.

Task	Built-in Wolfram Language	Repository at this snapshot
Ordinary local expansion	<code>Series</code> provides Taylor, Laurent, selected fractional-power and logarithmic expansions, including expansions at infinity [11].	The analytic path retains complete power-log blocks below an exclusive cutoff, with explicit local-variable and remainder information.
General asymptotic approximation	<code>Asymptotic</code> infers scales and accepts finite or infinite, real or complex centers [13].	Specialized real engines coexist with explicit native delegation and selected automatic routing; complete native coverage is still a recorded goal.
Formal series operations	<code>SeriesData</code> participates in arithmetic, composition, differentiation, and integration [12].	Analytic operations require the package’s contract; native wrappers deliberately refuse that promotion and expose “NativeResult” instead.
Series reversion	<code>InverseSeries</code> reverses appropriately structured <code>SeriesData</code> and permits a new inverse variable [14].	<code>AsymptoticInverse</code> adds real endpoint and branch models, generalized weights, specialized cores, retained construction data, and selected certification routes.

Task	Built-in Wolfram Language	Repository at this snapshot
Implicit solutions	<code>AsymptoticSolve</code> handles scalar or system problems, local solution centers, real-domain selection, and generated conditions [15].	The inverse APIs focus on admitted one-variable inverse models rather than being a replacement for the general implicit solver.
Special functions	Native expansion functionality already covers many special-function and non-Taylor examples [11, 13].	The documented real adapters add structured admission and error contracts for selected Bessel, Airy, error, incomplete Gamma, zeta, polylogarithmic, hypergeometric, elliptic, Gamma, and Barnes cases.
Lists and successive variables	Native <code>Series</code> threads over lists and accepts successive expansion specifications [11].	The native wrapper preserves these structures; that is not the same as implementing a new joint multivariate analytic germ algebra.
Numerical root evidence	Symbolic approximations and numerical solutions are distinct tasks.	<code>InverseNumericalCheck</code> supplies diagnostics, whereas <code>InverseCertificate</code> attempts an exact rational interval proof for its supported explicit expression.

3.3 What should not be claimed about the built-ins

It would be inaccurate to contrast this repository with a supposed Taylor-only native system. The official `Series` documentation includes fractional powers, logarithms, asymptotic examples, symbolic parameters, approximate input, and list threading. Likewise, `Asymptotic` is not restricted to power series: its documented inputs include integrals and solution expressions, and its scale can be inferred from the problem [11, 13].

It would also be inaccurate to equate the limitations of a single `SeriesData` exponent lattice with the expressiveness of all Wolfram expressions. Native symbolic powers or rapidly varying factors may remain outside a `SeriesData` object, and slow functions may occur in its coefficients. The package’s sparse generalized representation can be a much more convenient *contract and algorithmic interface* without implying that the native language cannot represent the same finite expression [12].

Conversely, native `AsymptoticSolve` should be part of an inverse-function comparison, not omitted in favor of `InverseSeries` alone. It documents Puiseux solutions, systems, real-domain restrictions, and solution centers. Its order is an approximation order, not necessarily polynomial degree [15]. An honest comparison must match the equation, selected local branch, target regime, retained scale, and actual error information.

3.4 The meaningful distinction is the result contract

A package analytic result generally carries a finite expression together with a local asymptotic remainder, assumptions, coordinate information, and construction-specific metadata. A native result instead preserves the backend output, records native provenance, and leaves package analytic remainder and exactness fields missing. In particular, `Normal` on a native result follows the stored native normalization and need not yield a finite polynomial [5].

For explicit native results, the stored request and expansion specifications are syntactic held records. The contract document does not claim that they are snapshots of all evaluated option values. Its ambient assumption record is likewise not the effective value of an explicit `Assumptions` option. Those are stated design boundaries, not new

metadata bugs. N01 concerns a different matter: the wrong engine is selected because the wrapper does not consult its advertised backend default.

Native `Asymptotic` has an asymptotic approximation contract; it should not be dismissed as mere formal coefficient manipulation. What it does not automatically supply to this wrapper is the package’s specific analytic remainder object, derivative contract, or rational root enclosure. Preserving that distinction is preferable to manufacturing a uniform-looking but unsupported result.

4 Mathematical examples that clarify the comparison

4.1 An exclusive cutoff and a native order are different requests

At this revision the analytic package call

```
AsymptoticExpansion[Exp[x], {x, 0, 3}]
```

returns the finite part

$$1 + x + \frac{x^2}{2}$$

with `PowerLogRemainder[x,3,0]`. Explicitly selecting native `Series` returns a stored native series whose finite part is

$$1 + x + \frac{x^2}{2} + \frac{x^3}{6}.$$

Both approximations are mathematically valid in their intended senses. They are not the same request contract. These particular finite parts were observed in the native evaluator, not merely inferred from documentation.

There is no universal “subtract one” conversion for all the package’s scales. A complete block can contain a polynomial in a logarithm; other engines count correction blocks after extracting a carrier or an exact core. A comparison should therefore report the returned powers and remainder, rather than comparing only the integer supplied in the input.

For an ordinary analytic power-log object, the relevant form is

$$s(x) = \sum_{\alpha < \beta} x^\alpha P_\alpha(\log x) + O\left(x^\beta (1 + |\log x|)^k\right), \quad x \longrightarrow 0^+. \quad (1)$$

The local coordinate can be a displacement or reciprocal source coordinate. The finite expression alone does not determine the remainder class or the branch on which it is valid. Consequently, a default that unexpectedly switches between an analytic result and a native result is not a cosmetic formatting problem.

4.2 A generalized inverse with an independent coefficient oracle

Consider, for a fixed real $p > 1$,

$$y = x + x^p, \quad x > 0, \quad y \longrightarrow 0^+.$$

The derivative $1 + px^{p-1}$ is positive, so the small positive inverse is unambiguous. Put $t = y^{p-1}$ and $x = yW(t)$. The inverse equation becomes

$$W(t) + tW(t)^p = 1, \quad W(0) = 1. \quad (2)$$

Writing $W = 1 + v$, the equation is $v = -t(1 + v)^P$. Lagrange inversion gives, for $n \geq 1$,

$$[t^n]v = \frac{(-1)^n}{n} [z^{n-1}](1 + z)^{nP} \quad (3)$$

$$= \frac{(-1)^n}{n} \binom{np}{n-1} = \frac{(-1)^n}{n!} \prod_{j=0}^{n-2} (np - j). \quad (4)$$

The product is empty when $n = 1$. Thus

$$x = y - y^p + p y^{2p-1} - \frac{p(3p-1)}{2} y^{3p-2} + O(y^{4p-3}). \quad (5)$$

The implicit-function theorem applied at $(t, W) = (0, 1)$ justifies a convergent local expansion in t ; substituting positive $t = y^{p-1}$ gives the stated real asymptotic interpretation for fixed p .

For $p = \sqrt{2}$, the powers are $1, \sqrt{2}, 2\sqrt{2} - 1, 3\sqrt{2} - 2, \dots$. They do not lie in a single rational exponent lattice in y . Nevertheless, this one-gap example also has the ordinary auxiliary-variable formulation (2). That observation prevents an exaggerated claim that irrational powers necessarily require a wholly different symbolic language. What the package makes convenient is keeping the real chart, exact weights, complete blocks, and error information in one reusable object.

The supplied independent oracle constructs the coefficients above using exact rational arithmetic and substitutes them into (2). It checks 50 combinations: five rational values $p > 1$, each through formal orders 1–10. Every residual coefficient through the requested order vanishes. These checks validate the mathematical oracle, not the Wolfram package. No successful native execution of this comparison is asserted here. For an ordinary integer-power comparison, $p = 2$ yields $y - y^2 + 2y^3 - 5y^4 + 14y^5 + \dots$, which is a natural `InverseSeries` differential fixture.

4.3 Why the certificate implementation is substantively different

The inspected certificate code separates approximate seed selection from the subsequent proof. Its arithmetic uses exact rational endpoints and directed dyadic rounding. Elementary exponential and logarithmic enclosures use explicit series tails. This is a materially stronger kind of evidence than a small floating-point residual, within the supported expression language [7].

The elementary argument behind the successful certificate path is worth making explicit. Suppose f is continuous on a closed interval I , its derivative is enclosed by D with $0 \notin D$, and $c \in I$. Let R enclose $f(c) - y$, let $\varepsilon = \max |R|$, and let $\mu = \inf_{d \in D} |d| > 0$. If

$$B = [c - \varepsilon/\mu, c + \varepsilon/\mu] \subseteq I,$$

monotonicity and the residual bound imply an opposite-sign bracket inside I and hence a unique root there. The implementation can alternatively establish existence from enclosed endpoint signs. Once existence is known, the mean-value theorem yields the sharper enclosure

$$J = I \cap B \cap (c - R/D).$$

The distance from c to the furthest endpoint of J bounds the root error.

The code also checks retained source-domain conditions on the whole closed verification interval. Its declared scope is a unique root of the stored explicit expression in that interval, not every function compatible with an unspecified input remainder and not a global inverse-branch theorem. These are good distinctions to preserve. They do not make every native symbolic approximation certifiable, nor does this inspection independently establish correctness of the whole certificate implementation.

5 N01: configured backend defaults are ignored

Reproduced request-contract defect

Changing `Options[AsymptoticExpansion]` with `SetOptions` does not change the dispatcher's implicit backend. A configured native-series request still takes the analytic package path and omits the cubic term in the example.

5.1 Witness and observed result

In a fresh kernel after loading the pinned package:

```
a = AsymptoticExpansion[Exp[x], {x, 0, 3},
  "Backend" -> "Series"];
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
b = AsymptoticExpansion[Exp[x], {x, 0, 3}];
{Options[AsymptoticExpansion, "Backend"],
 {a["Kind"], Normal[a]}, {b["Kind"], Normal[b]}}
```

The recorded result is

```
{{"Backend" -> "Series"},
 {"Native", 1 + x + x^2/2 + x^3/6},
 {"Forward", 1 + x + x^2/2}}
```

The option table was changed successfully. The failure is not in `SetOptions`, which is documented to change a symbol's default option settings [16]. It is in the wrapper's consumption of that table.

The baseline observation also set `"Backend" -> "Series"` on `AsymptoticExpand` and obtained the same package result. The alias copies the main option table during loading but forwards its arguments directly to the main function; its independently changed option table is not consulted.

5.2 Root cause

In `expansionDispatch`, near lines 67–82 of `src/Kernel/NativeCompatibility.wl`, the choice is:

```
backend = If[values === {}, Automatic,
  ReleaseHold[First[values]]];
```

Here `values` is extracted from explicitly supplied selector rules. An absent selector is replaced with the literal `Automatic`, not with the current default. The default option declaration and the implementation have become two independent sources of truth.

The same cause implies a second contract risk: a caller configuring the primary default to `"Package"` can still be routed by `Automatic`. For native-only inputs this can change refusal into a native wrapper. That consequence follows from the inspected dispatch path and the separately observed complex-input fallback; an additional direct probe for this combination did not complete because of a connector failure. It is therefore recorded as a source-derived consequence, not another executed witness.

5.3 Bounded repair and its actual validation

The primary-function repair is:

```
backend = If[values === {},
  OptionValue[AsymptoticExpansion, {}, "Backend"],
  ReleaseHold[First[values]]];
```

This preserves explicit first-selector precedence. It also avoids consuming a delayed default when an explicit selector is present. The accompanying regression specification covers delayed-default evaluation and invalid defaults, but those complete specifications should not be mistaken for executed observations.

An equivalent edit was applied to the pinned standalone source in a native kernel. The source anchor occurred once. Before changing the option, the candidate returned the usual forward quadratic expression. After setting the primary default to "Series", both the primary call and the held alias returned the native cubic expression. Explicitly supplying "Backend" -> Automatic still returned the forward quadratic expression. These four observations establish the narrow behavior change, not general native compatibility acceptance.

5.4 The alias decision must remain visible

There are two coherent policies. Under a true-alias policy, configuration is owned by `AsymptoticExpansion`; users configure that symbol, and the alias inherits it. Under independent-entry policies, each public head owns its defaults and the dispatcher must retain the entry symbol through preparation. What is not coherent is advertising a mutable alias option table whose changes silently do nothing.

The supplied candidate implements only the first policy's effective primary backend lookup. It does not remove the alias's misleading independent table or implement independent alias defaults. The preferred longer-term repair is to decide and document ownership, then pass that owner explicitly to option resolution. This is a small request-boundary change, not a reason to refactor the mathematical engines.

6 N02: selector discovery depends on unrelated input syntax

Reproduced programmatic-input defect

With `key = "Backend"`, a literal specification and a stored specification can give different outcomes for the same expansion. The former leaks the wrapper selector into native-option classification and fails.

6.1 Minimal contrast

```
key = "Backend";
spec = {x, 0, 3};
a = AsymptoticExpansion[Exp[x], {x, 0, 3}, key -> "Series"];
b = AsymptoticExpansion[Exp[x], spec, key -> "Series"];
```

The first result was:

```
Failure["NativeOptionConflict", <|
  "MessageTemplate" ->
    "No native backend supports all the supplied option keys.",
  "Options" -> {HoldComplete["Backend"]}]>]
```

The second result had "Kind" -> "Native". Explicit literal "Backend" -> "Series" also worked, as observed in N01. Thus the failure is not lack of support for native series, and it is not an invalid backend value.

6.2 Why the literal and computed specifications diverge

The wrapper is held. Its early selector search recognizes a rule whose left side is already the literal string "Backend". Meanwhile, `nativeComputedArgumentQ` returns false for an immediate or delayed rule, regardless of whether the rule's key is computed. It also returns false for a literal specification beginning with an unassigned variable.

In the literal-specification case, no preparation is requested. The held key symbol is missed during selector discovery and stripping. Later ordinary argument evaluation resolves it to "Backend", but dispatch has already moved past the point where that selector belongs. The native-option checker then sees the wrapper-only key and produces the conflict.

A stored specification is recognized as computed, so the trailing arguments are prepared before backend selection. That preparation also evaluates the option key. The selector is then visible in time and is stripped correctly. The exact same option program therefore behaves differently because an unrelated specification was written indirectly.

6.3 Repair without releasing option values

The candidate moves selector discovery until after any existing preparation, normalizes keys of recognized option rules, and only then performs discovery and stripping. A key helper is:

```
nativeResolveOptionKeys[HoldComplete[Rule[key_, value_]]] :=
  With[{resolved = ReleaseHold[HoldComplete[key]]},
    HoldComplete[Rule[resolved, value]]];

nativeResolveOptionKeys[
  HoldComplete[RuleDelayed[key_, value_]]] :=
  With[{resolved = ReleaseHold[HoldComplete[key]]},
    HoldComplete[RuleDelayed[resolved, value]]];
```

The complete helper in the patcher traverses only recognized option containers. It does not walk into a rule's value or into the source program. It does not replace symbols indiscriminately throughout the original held request. The original-argument metadata is preserved separately.

After preparation, the dispatcher derives normalized parts, extracts selector values from those parts, and builds the cleaned request from the same representation. This is the important consistency requirement: selector recognition and selector removal must see identical normalized keys.

Native candidate observations returned the expected native cubic expression for the formerly failing literal-specification call. A nested delayed selector also returned a native result and incremented its selector-value counter exactly once. The complete supplied regression file additionally specifies computed-key counters, first-selector precedence, and noninterference with rules inside an option value. Those additional specifications were not all executed as a suite during this audit.

6.4 Why appending all defaults is not a safe shortcut

Neither N01 nor N02 should be repaired by blindly appending all entries of the public `Options` table to every call. The automatic router distinguishes *explicit* direction, branch, and resource constraints from omitted options. Materializing every default as an explicit rule would alter those tests and could force otherwise native requests back onto the package path. It would also change the documented ownership of omitted native defaults.

A correct request boundary must retain whether an option came from the caller, from a wrapper-owned default, or from the selected native engine. For the bounded patch, only backend-default lookup and option-key normalization change; the existing option-value and native-default policies remain in place.

7 N03: syntactic Function occurrence is not callable source role

Reproduced unnecessary refusal

An applied identity function around `Exp[I x]` prevents automatic native fallback, although the plain scalar expression succeeds and the built-in series handles the wrapped expression.

7.1 Witness

```
a = AsymptoticExpansion[Exp[I x], {x, 0, 3}];
b = AsymptoticExpansion[(# &)[Exp[I x]], {x, 0, 3}];
c = Series[(# &)[Exp[I x]], {x, 0, 3}];
```

The recorded values were:

```
a["Kind"]
(* "Native" *)

b
(* Failure["InexactInput", <|"MessageTemplate" ->
  "Exact real input is required; approximate and complex constants are rejected."
|>] *)

c
(* SeriesData[x, 0, {1, I, -1/2, -I/6}, 0, 4, 1] *)
```

This is an availability and request-classification defect. It is not evidence that the real analytic engine should accept complex coefficients or that a native complex result should acquire a real remainder theorem.

7.2 The overbroad protection predicate

The source's `automaticProtectedQ` searches the entire original source for `_Function`, alongside inverse functions, conditional expressions, series objects, and remainder objects. It repeats a similar search in the prepared request. That broad syntactic occurrence test conflates two roles: a top-level unapplied callable that the package is expected to apply, and a function expression used internally by a program that evaluates to an ordinary scalar source.

The distinction is visible even in the identity example. The complete held source contains a `Function`, but the result to be expanded is not an unapplied function. Sending the request directly to the package path prevents the

later eligible `InexactInput` failure from reaching native fallback.

7.3 Narrow the Function test, not the branch policy

The candidate removes `_Function` from the two recursive protection searches and adds explicit top-level tests:

```
MatchQ[First[nativeHeldArguments[original]],
  HoldComplete[_Function]] ||
MatchQ[First[nativeHeldArguments[request]],
  HoldComplete[_Function]]
```

The deep searches for inverse functions, conditions, package series, remainders, and the prepared callable marker remain. Explicit direction, branch, and resource-option protections also remain. This is deliberately not “remove all protection and always try another backend.”

On the temporary native candidate, the identity-wrapped complex source returned the native cubic expansion. The genuine unapplied callable `Exp[##]` & still returned the analytic forward quadratic expression. Adding `"MaxTerms" -> 100` to the wrapped complex source still forced the package path and returned its real-input refusal, preserving the explicit resource contract.

One exploratory candidate call used an unrestricted quadratic `InverseFunction`. The kernel issued its multivalued-inverse warning and returned the other local branch. Since that call lacked a branch condition and no matched baseline branch comparison was completed, it is *not* counted as a successful inverse-branch regression. Its returned value and warning remain in the evidence record rather than being silently discarded. The supplied desired-behavior tests use explicit guard checks for inverse and conditional sources; the repository’s existing inverse-syntax tests should also be run before integrating the candidate.

7.4 The right equivalence claim is deliberately limited

The useful invariant is that a side-effect-free scalar normalization such as applying the identity function must not change admission to native fallback. It is not that every two programs with the same eventual value must have identical evaluation traces. The native-contract document explicitly allows computed trailing containers to be prepared before the source and does not promise arbitrary direct-native side-effect ordering [5].

A future classifier should retain the source’s semantic role through preparation: scalar expression, unapplied callable, inverse request, conditional source, existing analytic object, or native structure. That small classification record is more robust than treating the presence of a syntax head anywhere in the original expression as a permanent restriction.

8 N04: quadratic validation of nested option containers

Measured structural performance issue

A single option wrapped in increasingly deep lists causes selector discovery to revalidate the same subtrees. At depths 4, 8, 16, and 32, the observed `nativeOptionTreeQ` entry counts were 14, 44, 152, and 560.

8.1 Source mechanism and exact recurrence

The relevant definitions have the structure:

```
nativeOptionTreeQ[HoldComplete[_Rule | _RuleDelayed]] := True;
nativeOptionTreeQ[HoldComplete[(List | Sequence)[args____]] :=
  And @@ (nativeOptionTreeQ /@
    nativeHeldArguments[HoldComplete[args]]);

nativeSelectorValues[held : HoldComplete[List[args____]]] /;
  nativeOptionTreeQ[held] :=
  Flatten[nativeSelectorValues /@
    nativeHeldArguments[HoldComplete[args]], 1];
```

Take one non-selector rule, such as `Assumptions -> True`, surrounded by d singleton lists. Validation at the outer selector node visits d list nodes plus the rule. Recursive selector discovery then validates the depth $d - 1$ subtree again, and so on. The total number of predicate entries is

$$T(d) = \sum_{j=1}^d (j + 1) = \frac{d(d + 3)}{2}. \quad (6)$$

The input contains only one rule, and the selector result is empty. The repeated work is therefore not justified by a large output or by many candidate mathematical coefficients.

Nesting depth	Input tree nodes	Predicted validations	Observed validations
4	5	14	14
8	9	44	44
16	17	152	152
32	33	560	560

The native measurement inserted a counter at entry to `nativeOptionTreeQ`; it did not change the returned predicate values. The supplied profiling script restores the original downvalues afterwards. The measurement is for *selector discovery only*. It is not a whole-call runtime benchmark, a peak-memory measurement, or a claim that nesting dominates ordinary workloads. Similar guarded traversals occur elsewhere in the option helpers, but this report does not multiply the measured count into an invented total-call figure.

The independent Python structural check verifies the closed recurrence for 65 depths, including depth zero. That model check is separate from the four native measurements.

8.2 A linear structural plan

The natural repair has two linear structural passes, rather than a recursive validator nested inside every recursive consumer. First, classify the held syntax bottom-up, recording whether each node is a rule, a valid option container, or ordinary data. Do not evaluate keys or values in this pass. Second, emit the normalized option representation only for subtrees admitted as option containers. During emission, normalize each key once, retain held values, collect selector information, and construct the cleaned tree.

The separation avoids a subtle side-effect trap. A list can contain a rule without being an option container. Eagerly evaluating the rule's key while classifying such a list would perform an option-related side effect in data that should never have been treated as options. Pure classification before materialization prevents that error.

Both passes visit each relevant syntax node a bounded number of times. The linear claim concerns structural traversal; user-defined key programs and native option evaluation can still have arbitrary cost. Keep any annotations request-local. A global cache keyed only by syntax would be inappropriate when keys or values depend on mutable definitions or evaluation context.

The bounded N02 candidate deliberately reuses the existing recursive validation helpers. It therefore *does not* implement this performance repair and may add another such traversal. Fixing the semantic bug first is reasonable, but its cost should not be described as linear until this separate parser change is made and measured.

9 N05: a current README contradiction

The README introduction says automatic native routing is implemented. The current dispatcher and the complex-source observation agree. Its final “Development status” paragraph, however, still says that both Automatic and “Package” use the same package path and describes automatic fallback as required future work. That paragraph also emphasizes the older explicit-mode milestone where the introduction describes the later automatic milestone [1, 4].

A replacement should state the implemented policy without overclaiming coverage:

AsymptoticExpand is a held alias of AsymptoticExpansion. Automatic retains successful analytic package requests and routes selected native specifications, native options, and representation failures to a native backend. “Package” disables that routing. Complete native input coverage remains a development target; the validation record identifies the scope and snapshot of each focused run.

The archive includes this replacement as a documentation proposal. No new passing count is inserted. Counts tied to older source snapshots should remain in the validation record with their original scope rather than being used as a general current correctness statement.

10 Repair design and development opportunities arising from this audit

10.1 A small request record instead of another mathematical refactor

N01–N04 share a boundary problem: the wrapper repeatedly rediscovers information from syntax after different pieces have been evaluated. A compact internal request record would eliminate that ambiguity without changing coefficient calculus. The record needs an original held request, a prepared source, source role, ordered specifications, normalized held options, selector presence and value, and the owner of wrapper defaults.

There should be one authoritative transition sequence:

1. Preserve the original held request and establish the public entry symbol.
2. Prepare only the argument containers admitted by the existing policy.
3. Classify option containers, then normalize their keys without consuming values.
4. Resolve the first explicit selector or the appropriate wrapper-owned default.
5. Classify the source role and select the analytic or native path.
6. Construct a result with the selected path’s actual contract and provenance.

This is a proposal for the dispatch boundary only. It does not replace the existing distinction between analytic and native results, introduce an unrequested global registry, or solve the already recorded mathematical coverage obligations.

10.2 Make option ownership explicit

The union of native and package option names is useful for discovery, but a flat `Options` table does not explain who owns each omitted value. "Backend" is unambiguously owned by the wrapper. Explicit native options belong to the chosen native call. Omitted native options currently retain native defaults under the documented direct-native contract. Package assumptions and resource options have their own captured or explicit policies. An implementation should not infer all of these meanings from one merged list.

The next API decision should specify which defaults the alias owns and which symbol users configure. Under independent-entry ownership, pass the entry symbol through every preparation continuation and use it for wrapper-owned `OptionValue` lookups. Under true-alias ownership, document the primary symbol as the configuration owner and remove or clearly constrain the misleading independent configuration surface. Either choice should be tested for omitted, explicit, delayed, and invalid selectors.

It is also worth distinguishing an explicit option from an omitted option whose resolved default happens to have the same value. The existing router uses explicit presence to protect branch and resource contracts. A normalized record should retain that provenance rather than losing it when producing ordinary rules.

10.3 Metamorphic tests for request semantics

Current native tests already compare many mathematical outputs and retained contracts. The new witnesses suggest a complementary family of metamorphic tests: transformations of a request that should preserve a specified portion of its meaning.

For an effective wrapper default, explicitly supplying that value and omitting it after configuring the owner should agree on the selected backend and result contract. A literal selector key and an alias evaluating to that key should agree, independent of whether the expansion specification was stored in a variable. Wrapping a scalar source in a side-effect-free identity application should not change native fallback eligibility. Nesting options in additional valid containers should preserve ordered first-selector precedence. In contrast, adding an explicit branch or resource restriction is not an equivalence transformation: it must retain its restrictive effect.

Comparisons must exclude provenance fields that are intentionally syntactic. Two requests with different original syntax should not be required to have identical `OriginalArguments`. Their normalized native result or selected analytic contract can agree while that historical record differs. Likewise, callback-count expectations should distinguish wrapper-owned selectors from delayed option values evaluated by a native engine. The wrapper should not promise a universal native callback count that the backend itself does not guarantee.

10.4 Focused acceptance and performance workloads

The supplied `NovelRegressions.wlt` contains 17 desired-behavior tests. They cover primary backend defaults, explicit overrides, alias inheritance under the bounded policy, selector-key aliases, nested delayed selectors, source-program replay, genuine callables, and retained protection guards. The file is a specification supplied for execution, not a claim of 17 passing native tests in this audit.

Before integration, also run the repository's selected native-automatic, native-contract, presentation, assumption-replay, and inverse-syntax tests. The existing `CheckNativeAutomatic.wl` is an appropriate focused entry point. This recommendation does not require or claim a full-suite run. A selected upstream `NativeAutomatic.wlt` invocation attempted here did not return a report because of a connector error.

For N04, benchmark two axes separately: deeply nested singleton option lists and wide, shallow lists containing many option rules. Record structural visit counts and elapsed time separately, and include the cost of evaluating

no user callbacks. Then add callback-bearing keys as a semantic-control workload, not as a basis for claiming parser speedups. A parser optimization should reduce repeated structural work while keeping callback counts, ordering, cleaned requests, refusals, and original-request metadata unchanged.

No matched whole-package timing study was completed here. Consequently, this report makes no claim that the package is faster or slower than native `Series`, `Asymptotic`, or `InverseSeries` in general. Such a comparison would have to match achieved order and error information, not merely input option values.

10.5 Priorities

First, repair primary default consumption, key normalization, and the callable-role predicate as separate reviewable changes. Second, settle alias default ownership so the primary fix is not mistaken for complete API closure. Third, replace repeated option-tree validation with bounded linear structural passes and measure that specific workload. Update the stale README paragraph alongside these changes. The existing register remains the authority for other correctness repairs and broader mathematical development; this report does not replace its priorities with a second copy of the same roadmap.

11 Artifacts, execution evidence, and reproduction

11.1 What the archive contains

File	Purpose and evidence boundary
article/asymptotic-audit.tex and PDF	This article; the TeX source is self-contained and builds with standard TeX Live packages.
code/patch_native_dispatch.py	Non-destructive staging of three bounded edits; refuses missing or duplicate anchors and existing output paths.
code/NovelRegressions.wlt	Seventeen desired-behavior native specifications; baseline failures are expected, complete candidate execution not claimed.
code/RunAudit.wl	Focused runner with nonempty-report, abort, export, and entry-file-change checks. No upstream full suite is selected.
code/ProfileOptionNesting.wl	Request-parser predicate instrumentation, restoring the original downvalues afterwards.
code/test_patch_native_dispatch.py	Thirteen executed Python tests of source-excerpt patch fixtures and refusal behavior. Not package execution.
code/independent_oracles.py	Exact rational inverse-identity and structural-recurrence checks, independent of the package.
evidence/native_observations.json	Transcriptions of successful native observations, patch scope, and exploratory-control qualifications.
evidence/novelty_ledger.json	Relation of each new item to the existing review register and its specific new evidence.

The Python test log records 13 successful fixture-test methods. The independent oracle record contains 50 exact inverse-identity cases and 65 parser-recurrence cases. These populations are not added to native observations

to manufacture a package acceptance total. The article build and visual PDF check are artifact validation, not mathematical or package-test evidence.

11.2 Stage and test a temporary standalone candidate

For a local copy of the pinned root standalone:

```
python code/patch_native_dispatch.py \
/path/to/AsymptoticAnalysis.wl \
/path/to/new-candidate.wl \
--diff /path/to/new-candidate.diff

wolframscript -file code/RunAudit.wl \
/path/to/new-candidate.wl \
/path/to/new-candidate-results.json
```

The backslash line continuations above are shell notation; in PowerShell use a single command line or its normal continuation syntax. Every output path must be new. The stager does not fetch a repository, alter the original file, or assert that matching source anchors prove a whole snapshot's identity. Verify the input revision independently before staging.

Equivalent edits were applied to a temporary generated standalone in the native evaluator, and the main witnesses and selected controls were observed there. The distributed Python staging tool itself was exercised locally on transcribed source-excerpt fixtures, not against a complete locally downloaded checkout. Its anchor checks intentionally refuse unexpected source layouts. The integrated candidate still needs the focused native suites described above.

For maintenance, apply the reviewed changes to the canonical modular source, then regenerate the standalone using the repository's existing builder. A repair made only to the generated root file will be overwritten by a later build. Do not load the isolated companion module as though it were a complete package.

11.3 Reproduce the independent checks and article

```
python -m unittest discover -s code -p "test_*.py" -v
python code/independent_oracles.py --output new-oracle-results.json

cd article
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-audit.tex
pdflatex -interaction=nonstopmode -halt-on-error asymptotic-audit.tex
```

The oracle refuses to overwrite an existing output. The provided results record the checks already executed when preparing this archive. No font files, repository binaries, or checksum files are needed to use the deliverable.

11.4 Final assessment

The repository's useful distinction from the native system is its retained analytic structure and specialized inverse machinery, not a blanket absence of comparable native mathematics. Its native wrapper correctly makes room for results outside the analytic real representation without pretending to have proved more than it has. The new defects lie in the request language that chooses between those contracts.

The repair should therefore preserve the mathematical engines and the analytic/native distinction, while

making backend-default consumption, option-key normalization, and source-role classification consistent. The counterexamples here are small enough to become stable regression fixtures; the parser recurrence is precise enough to give a meaningful performance acceptance condition. That is a more actionable next increment than another broad catalogue of the already recorded issues.

References

- [1] Vladimir Reshetnikov and contributors. *AsymptoticAnalysis repository README*, pinned revision 6687962f3c858a4f93623cfc496f33e35c6763d4. *README.md*. Accessed 9 September 2026.
- [2] *Modular kernel implementation and source map*. `src/Kernel/README.md`.
- [3] *Public declarations, assumption boundary, and core implementation*. `src/Kernel/AsymptoticAnalysis.wl`.
- [4] *Native backend selection and result construction*. `src/Kernel/NativeCompatibility.wl`. Key locations: `option-tree helpers`, `expansionDispatch`, `automaticProtectedQ`, and `nativeExpansion`.
- [5] *Native expansion result contracts*. `docs/development/NATIVE_RESULT_CONTRACTS.md`.
- [6] *Ordinary series arithmetic and held normalization*. `src/Kernel/SeriesArithmetic.wl`.
- [7] *Exact rational inverse certificates*. `src/Kernel/InverseCertificates.wl`.
- [8] *Code-review report indexes*. `external-reports/code-review/README.md`; `external-reports/code-review/wave-1/README.md`; `external-reports/code-review/wave-2/README.md`.
- [9] *Code-review implementation status and complete finding crosswalk*. `docs/development/CODE_REVIEW_STATUS.md`.
- [10] *Automatic native routing tests and focused validation entry*. `src/Tests/NativeAutomatic.wlt`; `validation/CheckNativeAutomatic.wl`; `validation/FocusedTests.wl`.
- [11] Wolfram Research. *Series*. Official Wolfram Language reference. <https://reference.wolfram.com/language/ref/Series.html>. Accessed 9 September 2026.
- [12] Wolfram Research. *SeriesData*. Official reference. <https://reference.wolfram.com/language/ref/SeriesData.html>. Accessed 9 September 2026.
- [13] Wolfram Research. *Asymptotic*. Official reference. <https://reference.wolfram.com/language/ref/Asymptotic.html>. Accessed 9 September 2026.
- [14] Wolfram Research. *InverseSeries*. Official reference. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Accessed 9 September 2026.
- [15] Wolfram Research. *AsymptoticSolve*. Official reference. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>. Accessed 9 September 2026.
- [16] Wolfram Research. *SetOptions*. Official reference. <https://reference.wolfram.com/language/ref/SetOptions.html>. Accessed 9 September 2026.